

Interview Practice

Thirty classic quant-researcher questions on stochastic calculus and derivatives pricing, from warm-ups to hard ones. Each has a hint, a full derivation, a link to the section that explains the idea, and a short simulation that confirms the answer.

Practise as in an interview: say your reasoning out loud, derive on paper before opening the hint, and check the answer's limiting cases (what happens as the drift goes to 0, or as time grows?). Then run the check. Being able to verify an answer yourself is half of what interviewers look for.

A · Brownian motion

A1 The fourth moment easy

Let W be a standard Brownian motion. Compute $\mathbb{E}[W_t^4]$.

Hint

Write $W_t = \sqrt{t} Z$ with $Z \sim N(0, 1)$. What are the moments of a standard normal?

Solution

$W_t = \sqrt{t} Z$, so $\mathbb{E}[W_t^4] = t^2 \mathbb{E}[Z^4]$. From the moment generating function $\mathbb{E}[e^{uZ}] = e^{u^2/2} = \sum_k \frac{u^{2k}}{2^k k!}$, the coefficient of $u^4/4!$ gives $\mathbb{E}[Z^4] = \frac{4!}{2^2 2!} = 3$. Hence $\boxed{\mathbb{E}[W_t^4] = 3t^2}$.

Follow-up: $\mathbb{E}[Z^{2k}] = (2k-1)!!$, so $\text{Var}(W_t^2) = 3t^2 - t^2 = 2t^2$. That is where the " $2t^2/n$ " in section 2.1 comes from.

Related: [1.2 Defining Properties](#)

Show the code (Python + NumPy)

```
import numpy as np
t = 2.0
W = np.random.default_rng(101).normal(0, np.sqrt(t), 2_000_000)
print(f"E[W_t^4] simulated {np.mean(W ** 4):.3f}    formula 3t^2 = {3 * t ** 2:.3f}")
```

Output when we ran it:

```
E[W_t^4] simulated 11.989    formula 3t^2 = 12.000
```

A2 Both times positive medium

Compute $P(W_1 > 0, W_2 > 0)$.

Hint

Write $W_2 = W_1 + (W_2 - W_1)$: two independent standard normals X, Y . Think about angles in the (X, Y) plane.

Solution

With $X = W_1$ and $Y = W_2 - W_1$ independent $N(0, 1)$, the pair (X, Y) is rotation-invariant, so a probability is an angle over 2π . $\{X > 0\}$ is a half-plane and $\{X + Y > 0\}$ is another, whose boundary makes an angle of $\pi/4$ with the first. Their intersection is a wedge of angle $\pi - \pi/4 = 3\pi/4$, so the probability is $\frac{3\pi/4}{2\pi} = \boxed{3/8}$.

General rule: for standard bivariate normals with correlation ρ , $P(\text{both} > 0) = \frac{1}{4} + \frac{\arcsin \rho}{2\pi}$. Here $\rho = \text{Corr}(W_1, W_2) = 1/\sqrt{2}$, so $\arcsin \rho = \pi/4$.

Related: [1.2 Defining Properties](#)

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(102)
W1 = rng.normal(size=2_000_000); W2 = W1 + rng.normal(size=W1.size)    # W_2 = W_1 +
independent increment
print(f"P(W_1 > 0, W_2 > 0) simulated {np.mean((W1 > 0) & (W2 > 0)):.4f}    formula 3/8 = {3
/ 8:.4f}")
```

Output when we ran it:

```
P(W_1 > 0, W_2 > 0) simulated 0.3753    formula 3/8 = 0.3750
```

A3 The time average medium

What is the distribution of $\int_0^1 W_t dt$?

Hint

It is a limit of sums of jointly normal variables. Compute its variance using $\mathbb{E}[W_s W_t] = \min(s, t)$.

Solution

A limit of linear combinations of Gaussians is Gaussian, with mean $\int_0^1 \mathbb{E}[W_t] dt = 0$. Its variance is

$$\int_0^1 \int_0^1 \min(s, t) ds dt = 2 \int_0^1 \int_0^t s ds dt = 2 \int_0^1 \frac{t^2}{2} dt = \frac{1}{3}. \text{ So } \boxed{\int_0^1 W_t dt \sim N(0, 1/3)}.$$

Second route: integration by parts (question C5) gives $\int_0^1 W_t dt = \int_0^1 (1 - s) dW_s$, and the Itô isometry gives $\int_0^1 (1 - s)^2 ds = 1/3$.

Related: [1.2 Defining Properties](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, n, steps = np.random.default_rng(103), 20_000, 500
W = np.cumsum(rng.normal(0, np.sqrt(1 / steps), (n, steps)), axis=1)
I = W.mean(axis=1) # Riemann sum for the integral
of W over [0, 1]
print(f"mean {I.mean():+.4f} (0)   variance {I.var():.4f}   formula 1/3 = {1 / 3:.4f}")
```

Output when we ran it:

```
mean -0.0031 (0)   variance 0.3317   formula 1/3 = 0.3333
```

A4 The Brownian bridge medium

Given $W_1 = 0$, what is the distribution of W_t for $0 < t < 1$?

Hint

(W_t, W_1) is jointly normal. Use the conditional-normal formula.

Solution

$\text{Cov}(W_t, W_1) = t$ and $\text{Var}(W_1) = 1$, so $\mathbb{E}[W_t \mid W_1 = x] = tx$ and $\text{Var}(W_t \mid W_1) = t - t^2/1 = t(1 - t)$. With $x = 0$: $\boxed{W_t \mid W_1 = 0 \sim N(0, t(1 - t))}$, largest in the middle at $t = 1/2$.

Follow-up: on $[0, T]$ with $W_T = x$, the mean is tx/T and the variance $t(T - t)/T$. The process $W_t - tW_1$ is a Brownian bridge independent of W_1 . This is the construction behind section 1.3's zoom and 9.3's barrier correction.

Related: [9.3 Barrier Options](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, n, t = np.random.default_rng(104), 2_000_000, 0.3
Wt = rng.normal(0, np.sqrt(t), n); W1 = Wt + rng.normal(0, np.sqrt(1 - t), n)
near = np.abs(W1) < 0.01 # condition on W_1 = 0
(approximately)
print(f"Var(W_t | W_1 = 0) simulated {Wt[near].var():.4f} formula t(1 - t) = {t * (1 - t):.4f}")
```

Output when we ran it:

```
Var(W_t | W_1 = 0) simulated 0.2101 formula t(1 - t) = 0.2100
```

A5 The expected maximum medium

Compute $\mathbb{E}[\max_{s \leq t} W_s]$.

Hint

The reflection principle (section 2.3) gives the distribution of the maximum.

Solution

By reflection, $P(M_t \geq a) = 2P(W_t \geq a) = P(|W_t| \geq a)$ for $a \geq 0$, so M_t has the same distribution as $|W_t|$. Therefore $\mathbb{E}[M_t] = \mathbb{E}|W_t| = \sqrt{t} \mathbb{E}|Z| = \boxed{\sqrt{2t/\pi}}$.

Follow-up: a simulation on a time grid *underestimates* the maximum, because it misses peaks between grid points. The bias is about $0.5826 \sqrt{\Delta t}$, the same constant as the barrier continuity correction in 9.3.

Related: [2.3 Reflection Principle](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, n, steps, t = np.random.default_rng(105), 10_000, 1_000, 1.0
M = np.maximum(np.cumsum(rng.normal(0, np.sqrt(t / steps), (n, steps))),
axis=1).max(axis=1), 0)
print(f"E[max W] simulated {M.mean():.4f} formula sqrt(2t/pi) = {np.sqrt(2 * t / np.pi):.4f} (grid slightly underestimates)")
```

Output when we ran it:

$E[\max W]$ simulated 0.7718 formula $\sqrt{2t/\pi} = 0.7979$ (grid slightly underestimates)

B · Hitting times and stopping

B1 Which barrier first? easy

Starting at 0, what is the probability that Brownian motion hits $+a$ before $-b$ ($a, b > 0$)?

Hint

Stop W at the exit time τ and use that it is a martingale.

Solution

$W_{t \wedge \tau}$ is a bounded martingale, so optional stopping gives $\mathbb{E}[W_\tau] = 0$. With $p = P(\text{hit } a \text{ first})$:

$$pa - (1 - p)b = 0, \text{ so } p = \frac{b}{a + b}.$$

Follow-up: the symmetric random walk (gambler's ruin, question B5) gives the same answer, since at integer levels it hits exactly.

Related: [2.2 Martingales](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, a, b, walks = np.random.default_rng(106), 3, 2, 20_000 # +-1 random walk
approximates BM exactly at integers
hits = 0
for _ in range(walks):
    x = 0
    while -b < x < a:
        x += 1 if rng.random() < 0.5 else -1
    hits += x == a
print(f"P(hit +{a} before -{b}) simulated {hits / walks:.4f}    formula b/(a+b) = {b / (a + b):.4f}")
```

Output when we ran it:

P(hit +3 before -2) simulated 0.4068 formula $b/(a+b) = 0.4000$

B2 Expected exit time medium

Brownian motion starts at 0. What is the expected time until it leaves $(-a, b)$?

Hint

Which martingale involves t ? Combine it with the answer to B1.

Solution

$W_t^2 - t$ is a martingale, so $\mathbb{E}[W_\tau^2] = \mathbb{E}[\tau]$ (optional stopping, justified because τ has finite expectation). The upper level b is hit first with probability $\frac{a}{a+b}$, so $\mathbb{E}[W_\tau^2] = b^2 \frac{a}{a+b} + a^2 \frac{b}{a+b} = ab$. Hence $\mathbb{E}[\tau] = ab$.

Follow-up: for a one-sided barrier ($a \rightarrow \infty$) the expected time is infinite, even though the barrier is reached with probability 1.

Related: [2.2 Martingales](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, a, b, n, dt = np.random.default_rng(107), 1.0, 2.0, 20_000, 1e-3
x, t, alive = np.zeros(n), np.zeros(n), np.ones(n, bool)
while alive.any():
    x[alive] += np.sqrt(dt) * rng.normal(size=alive.sum()); t[alive] += dt
    alive &= (x > -a) & (x < b)
print(f"E[exit time of (-{a:.0f}, {b:.0f})] simulated {t.mean():.3f}    formula ab = {a * b:.3f}")
```

Output when we ran it:

```
E[exit time of (-1, 2)] simulated 2.059    formula ab = 2.000
```

B3 Which barrier first, with drift hard

$X_t = \mu t + W_t$. What is the probability that X hits $+a$ before $-b$?

Hint

Find λ such that $e^{\lambda X_t}$ is a martingale, then stop it.

Solution

$e^{\lambda X_t - (\lambda\mu + \lambda^2/2)t}$ is a martingale for every λ . Choosing $\lambda = -2\mu$ makes the time term vanish: $e^{-2\mu X_t}$ is a martingale. Optional stopping gives $p e^{-2\mu a} + (1 - p) e^{2\mu b} = 1$, so

$$p = \frac{1 - e^{2\mu b}}{e^{-2\mu a} - e^{2\mu b}}.$$

Check: as $\mu \rightarrow 0$, L'Hôpital's rule gives $b/(a + b)$, matching B1. With $\mu = 0.5$ and $a = b = 1$, $p \approx 0.731$.

Related: [6.2 Girsanov](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, mu, a, b, n, dt = np.random.default_rng(108), 0.5, 1.0, 1.0, 20_000, 1e-3
x, alive = np.zeros(n), np.ones(n, bool)
while alive.any():
    x[alive] += mu * dt + np.sqrt(dt) * rng.normal(size=alive.sum())
    alive &= (x > -b) & (x < a)
p = (1 - np.exp(2 * mu * b)) / (np.exp(-2 * mu * a) - np.exp(2 * mu * b))
print(f"P(hit +{a:.0f} before -{b:.0f}), drift {mu}: simulated {np.mean(x >= a):.4f}
      formula {p:.4f}")
```

Output when we ran it:

```
P(hit +1 before -1), drift 0.5: simulated 0.7371    formula 0.7311
```

B4 Expected hitting time with drift easy

$X_t = \mu t + W_t$ with $\mu > 0$. What is the expected time for X to reach $a > 0$?

Hint

$X_t - \mu t$ is a martingale (Wald's identity).

Solution

$X_t - \mu t = W_t$ is a martingale, so $\mathbb{E}[X_\tau] = \mu \mathbb{E}[\tau]$. Since $X_\tau = a$, $\boxed{\mathbb{E}[\tau] = a/\mu}$ (to be rigorous: stop at $\tau \wedge n$ and let $n \rightarrow \infty$ using $\mu > 0$).

Follow-up: by the drift version of the reflection principle, τ has the inverse Gaussian distribution. As $\mu \rightarrow 0$ its mean blows up, consistent with B2's follow-up.

Related: [2.2 Martingales](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, mu, a, n, dt = np.random.default_rng(109), 0.5, 2.0, 20_000, 1e-3
x, t, alive = np.zeros(n), np.zeros(n), np.ones(n, bool)
while alive.any():
    x[alive] += mu * dt + np.sqrt(dt) * rng.normal(size=alive.sum()); t[alive] += dt
    alive &= x < a
print(f"E[time to hit {a:.0f}] with drift {mu}: simulated {t.mean():.3f}    formula a/mu = {a / mu:.3f}")
```

Output when we ran it:

```
E[time to hit 2] with drift 0.5: simulated 4.040    formula a/mu = 4.000
```

B5 Gambler's ruin easy

A fair coin moves you ± 1 , starting at k . You stop at 0 or N . What is the chance of reaching N , and the expected number of flips?

Hint

Two martingales: the position S_n , and $S_n^2 - n$.

Solution

S_n is a martingale: $P(\text{reach } N) \cdot N = k$, so $P = k/N$. $S_n^2 - n$ is a martingale: $\mathbb{E}[S_\tau^2] = \mathbb{E}[\tau] + k^2$, and $\mathbb{E}[S_\tau^2] = N^2 \frac{k}{N} = Nk$. So $\mathbb{E}[\tau] = k(N - k)$.

Follow-up: with a biased coin ($p \neq \frac{1}{2}$), use the martingale $(q/p)^{S_n}$: $P = \frac{1-(q/p)^k}{1-(q/p)^N}$. That is the discrete version of B3.

Related: [1.1 Random Walk](#)

Show the code (Python + NumPy)

```

import numpy as np
rng, k, N, games = np.random.default_rng(110), 3, 10, 20_000
wins, lengths = 0, []
for _ in range(games):
    x, steps = k, 0
    while 0 < x < N:
        x += 1 if rng.random() < 0.5 else -1; steps += 1
    wins += x == N; lengths.append(steps)
print(f"P(reach {N}) {wins / games:.4f} (k/N = {k / N:.4f})    E[duration]
{np.mean(lengths):.2f} (k(N-k) = {k * (N - k)})")

```

Output when we ran it:

```

P(reach 10) 0.2980 (k/N = 0.3000)    E[duration] 20.93 (k(N-k) = 21)

```

C • Itô calculus

C1 Itô's lemma for W^3

easy

Compute $d(W_t^3)$.

Hint

Apply $df = f'(W) dW + \frac{1}{2} f''(W) dt$.

Solution

With $f(x) = x^3$: $f' = 3x^2$ and $f'' = 6x$, so $d(W_t^3) = 3W_t^2 dW_t + 3W_t dt$.

Follow-up: integrating gives $W_t^3 = 3 \int_0^t W^2 dW + 3 \int_0^t W ds$. Taking expectations, $0 = 0 + 0$, consistent with $\mathbb{E}[W_t^3] = 0$.

Related: [4.2 Itô's Lemma](#)

Show the code (Python + NumPy)

```

import numpy as np
rng, n, steps = np.random.default_rng(111), 20_000, 400
dt = 1 / steps
dW = rng.normal(0, np.sqrt(dt), (n, steps)); W = np.cumsum(dW, axis=1) - dW # W at left
ends
ito = np.sum(3 * W ** 2 * dW + 3 * W * dt, axis=1) # integral of 3W^2 dW + 3W dt
print(f"W_1^3 vs Ito rebuild on one path: {(W[0, -1] + dW[0, -1]) ** 3:.4f} vs
{ito[0]:.4f}")
print(f"average squared gap over {n:,} paths: {np.mean(((W[:, -1] + dW[:, -1]) ** 3 - ito)
** 2):.5f} (shrinks as steps grow)")

```

Output when we ran it:

```

W_1^3 vs Ito rebuild on one path: 0.4667 vs 0.5280
average squared gap over 20,000 paths: 0.02252 (shrinks as steps grow)

```

C2 Building a martingale medium

Show that $W_t^3 - 3tW_t$ is a martingale.

Hint

Compute its differential and check that the dt terms cancel.

Solution

From C1, $d(W^3) = 3W^2 dW + 3W dt$, and by the product rule $d(tW) = W dt + t dW$. So $d(W^3 - 3tW) = (3W^2 - 3t) dW + (3W - 3W) dt = (3W^2 - 3t) dW$. There is no drift, and the integrand is square-integrable, so it is a martingale.

Follow-up: these are the Hermite polynomials $H_n(W_t, t)$: W , $W^2 - t$, $W^3 - 3tW$, $W^4 - 6tW^2 + 3t^2$, and so on. They are the coefficients of the martingale $e^{\lambda W_t - \lambda^2 t/2}$ expanded in λ .

Related: [2.2 Martingales](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, s, t, n = np.random.default_rng(112), 0.5, 1.0, 2_000_000
Ws = 0.8 # condition on the value at
time s
Wt = Ws + rng.normal(0, np.sqrt(t - s), n)
M = lambda w, u: w ** 3 - 3 * u * w
print(f"E[W_t^3 - 3tW_t | W_s = {Ws}] simulated {M(Wt, t).mean():.4f} value at s {M(Ws,
s):.4f}")
```

Output when we ran it:

```
E[W_t^3 - 3tW_t | W_s = 0.8] simulated -0.6875 value at s -0.6880
```

C3 The exponential moment easy

Compute $\mathbb{E}[e^{\sigma W_t}]$ and use it to find $\mathbb{E}[S_T]$ for GBM.

Hint

Complete the square in the normal density, or use the moment generating function.

Solution

For $X \sim N(0, v)$, $\mathbb{E}[e^X] = e^{v/2}$, so $\boxed{\mathbb{E}[e^{\sigma W_t}] = e^{\sigma^2 t/2}}$. For $S_T = S_0 e^{(\mu - \sigma^2/2)T + \sigma W_T}$ this gives $\mathbb{E}[S_T] = S_0 e^{\mu T}$: the $-\sigma^2/2$ is exactly the Itô correction that makes the mean grow at rate μ .

Follow-up: the median is $S_0 e^{(\mu - \sigma^2/2)T}$, below the mean. The typical path underperforms the average (see 11.2).

Related: [5.1 Euler–Maruyama](#)

Show the code (Python + NumPy)

```
import numpy as np
sigma, t = 0.8, 2.0
W = np.random.default_rng(113).normal(0, np.sqrt(t), 4_000_000)
print(f"E[exp(sigma W_t)] simulated {np.mean(np.exp(sigma * W)):.4f} formula exp(sigma^2
t / 2) = {np.exp(sigma ** 2 * t / 2):.4f}")
```

Output when we ran it:

```
E[exp(sigma W_t)] simulated 1.8960 formula exp(sigma^2 t / 2) = 1.8965
```

C4 Variance of a stochastic integral medium

Compute $\text{Var} \left(\int_0^t W_s dW_s \right)$.

Hint

The Itô isometry, or the closed form of the integral.

Solution

The mean is 0 (it is a martingale). By the isometry, the variance is $\int_0^t \mathbb{E}[W_s^2] ds = \int_0^t s ds = \boxed{t^2/2}$.

Check: $\int_0^t W dW = \frac{1}{2}(W_t^2 - t)$ (section 3.1), so the variance is $\frac{1}{4} \text{Var}(W_t^2) = \frac{1}{4} \cdot 2t^2 = t^2/2$, using A1's follow-up.

Related: [3.3 Itô Isometry](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, n, steps, t = np.random.default_rng(114), 20_000, 250, 2.0
dt = t / steps
dW = rng.normal(0, np.sqrt(dt), (n, steps)); W = np.cumsum(dW, axis=1) - dW
I = np.sum(W * dW, axis=1) # Ito integral of W dW over [0, t]
print(f"Var(int_0^t W dW) simulated {I.var():.4f}    formula t^2/2 = {t ** 2 / 2:.4f}")
```

Output when we ran it:

```
Var(int_0^t W dW) simulated 2.0135    formula t^2/2 = 2.0000
```

C5 A deterministic integrand medium

What is the distribution of $\int_0^1 s dW_s$? Relate it to $\int_0^1 W_s ds$.

Hint

A stochastic integral of a *deterministic* function is Gaussian. For the relation, apply the product rule to sW_s .

Solution

With a deterministic integrand the integral is Gaussian with mean 0 and variance $\int_0^1 s^2 ds = 1/3$:

$$\int_0^1 s dW_s \sim N(0, 1/3).$$

The product rule $d(sW_s) = W_s ds + s dW_s$ (no extra term, because s has no randomness) integrates to $W_1 = \int_0^1 W_s ds + \int_0^1 s dW_s$. So $\int_0^1 W_s ds = \int_0^1 (1 - s) dW_s$, which is also $N(0, 1/3)$, matching A3.

Related: [4.4 Product rule](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, n, steps = np.random.default_rng(115), 20_000, 500
dt = 1 / steps
s = np.arange(steps) * dt
dW = rng.normal(0, np.sqrt(dt), (n, steps))
I = dW @ s # integral of s dW_s over [0, 1]
W1, intW = dW.sum(axis=1), (np.cumsum(dW, axis=1) * dt).sum(axis=1)
print(f"Var(int s dW) simulated {I.var():.4f} formula 1/3 = {1 / 3:.4f}")
print(f"integration by parts: max |int W ds - (W_1 - int s dW)| = {np.max(np.abs(intW - (W1 - I))):.4f} (discretization only)")
```

Output when we ran it:

```
Var(int s dW) simulated 0.3362 formula 1/3 = 0.3333
integration by parts: max |int W ds - (W_1 - int s dW)| = 0.0000 (discretization only)
```

C6 Solving the OU equation medium

Solve $dX_t = -\theta X_t dt + \sigma dW_t$ and give the mean and variance of X_t .

Hint

Multiply by the integrating factor $e^{\theta t}$.

Solution

$d(e^{\theta t} X_t) = e^{\theta t} (dX_t + \theta X_t dt) = \sigma e^{\theta t} dW_t$, so

$$X_t = X_0 e^{-\theta t} + \sigma \int_0^t e^{-\theta(t-s)} dW_s.$$

It is Gaussian with mean $X_0 e^{-\theta t}$ and variance (isometry) $\sigma^2 \int_0^t e^{-2\theta(t-s)} ds = \frac{\sigma^2}{2\theta} (1 - e^{-2\theta t})$.

Follow-up: the long-run distribution is $N(0, \sigma^2/2\theta)$ (section 5.2), and the half-life of the mean is $\ln 2/\theta$ (section 11.3).

Related: [5.2 A Zoo of Diffusions](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, theta, sigma, x0, t, n, steps = np.random.default_rng(116), 1.5, 0.8, 2.0, 1.0,
200_000, 400
dt, x = t / steps, np.full(n, x0)
for _ in range(steps):
    x += -theta * x * dt + sigma * np.sqrt(dt) * rng.normal(size=n)
print(f"mean {x.mean():.4f} (formula {x0 * np.exp(-theta * t):.4f})    variance "
      f"({x.var():.4f} "
      f"(formula {sigma ** 2 * (1 - np.exp(-2 * theta * t)) / (2 * theta):.4f}))")
```

Output when we ran it:

```
mean 0.4441 (formula 0.4463)    variance 0.2032 (formula 0.2027)
```

D • Martingales and change of measure

D1 The exponential martingale easy

For which b is $e^{aW_t - bt}$ a martingale?

Hint

Apply Itô's lemma and set the dt term to zero.

Solution

$d(e^{aW_t - bt}) = e^{aW_t - bt} (a dW_t + (\frac{1}{2}a^2 - b) dt)$. It is a martingale exactly when $b = a^2/2$.
Equivalently, $\mathbb{E}[e^{aW_t}] = e^{a^2 t/2}$ (C3).

Follow-up: this is the Girsanov density of section 6.2, and the object behind B3.

Related: [6.2 Girsanov](#)

Show the code (Python + NumPy)

```
import numpy as np
a, t = 1.3, 2.0
W = np.random.default_rng(117).normal(0, np.sqrt(t), 4_000_000)
for b in (a ** 2 / 2, 0.5):
    print(f"b = {b:.3f}: E[exp(aW_t - bt)] = {np.mean(np.exp(a * W - b * t)):.4f}    (a
martingale needs 1)")
```

Output when we ran it:

```
b = 0.845: E[exp(aW_t - bt)] = 0.9976    (a martingale needs 1)
b = 0.500: E[exp(aW_t - bt)] = 1.9889    (a martingale needs 1)
```

D2 A Girsanov expectation medium

Compute $\mathbb{E}[W_T e^{\theta W_T - \theta^2 T/2}]$.

Hint

Either differentiate the moment generating function in θ , or interpret the exponential as a change of measure.

Solution

Girsanov: the weight $Z_T = e^{\theta W_T - \theta^2 T/2}$ defines Q under which $W_t - \theta t$ is a Brownian motion. So $\mathbb{E}[W_T Z_T] = \mathbb{E}_Q[W_T] = \boxed{\theta T}$.

Direct check: $\mathbb{E}[W_T e^{\theta W_T}] = \frac{d}{d\theta} e^{\theta^2 T/2} = \theta T e^{\theta^2 T/2}$. Multiplying by $e^{-\theta^2 T/2}$ gives θT .

Related: [6.2 Girsanov](#)

Show the code (Python + NumPy)

```
import numpy as np
theta, T = 0.7, 1.5
W = np.random.default_rng(118).normal(0, np.sqrt(T), 4_000_000)
Z = np.exp(theta * W - theta ** 2 * T / 2) # Girsanov weight: W becomes BM
with drift theta
print(f"E[W_T Z] simulated {np.mean(W * Z):.4f}    formula theta T = {theta * T:.4f}")
```

Output when we ran it:

```
E[W_T Z] simulated 1.0494    formula theta T = 1.0500
```

D3 Why μ disappears easy

A stock has real-world drift μ . What is its forward price for delivery at T , and why does μ not appear?

Hint

Build the forward from the stock and a loan.

Solution

Buy the stock today for S_0 , financed by borrowing S_0 , and repay $S_0 e^{rT}$ at T . You then own the stock at T having paid $S_0 e^{rT}$, so no-arbitrage forces $F = S_0 e^{rT}$. A forward struck at K is worth $S_0 - K e^{-rT}$ today. This replication uses no model at all.

In risk-neutral language, under Q the stock grows at r ($dS = rS dt + \sigma S dW^Q$), because Q is defined by making discounted traded assets martingales. μ is a real-world quantity that the hedge removes (6.1, 6.2).

Related: [6.1 Black–Scholes by Hedging](#).

Show the code (Python + NumPy)

```
import numpy as np
from math import exp, sqrt
S0, r, mu, sigma, T, K = 100.0, 0.03, 0.12, 0.25, 1.0, 105.0
Z = np.random.default_rng(119).normal(size=2_000_000)
ST_Q = S0 * np.exp((r - sigma ** 2 / 2) * T + sigma * sqrt(T) * Z) # risk-neutral: drift r, not mu
print(f"forward price E_Q[S_T] {ST_Q.mean():.3f}    formula S0 e^(rT) = {S0 * exp(r * T):.3f}")
print(f"value of a forward struck at {K}: {exp(-r * T) * (ST_Q - K).mean():.3f}    formula S0 - K e^(-rT) = {S0 - K * exp(-r * T):.3f}")
```

Output when we ran it:

```
forward price E_Q[S_T] 103.047    formula S0 e^(rT) = 103.045
value of a forward struck at 105.0: -1.895    formula S0 - K e^(-rT) = -1.897
```

D4 Correlated Brownian motions easy

$W^{(1)}, W^{(2)}$ are Brownian motions with correlation ρ . What is the distribution of $W_t^{(1)} - W_t^{(2)}$?

Hint

Compute the variance of a difference.

Solution

It is Gaussian with mean 0 and variance $t + t - 2\rho t = \boxed{2(1 - \rho)t}$. Rescaled, $(W^{(1)} - W^{(2)}) / \sqrt{2(1 - \rho)}$ is itself a Brownian motion (Lévy's characterisation: continuous martingale, quadratic variation t).

Follow-up: this is the core of spread options and Margrabe's exchange-option formula, with volatility $\sqrt{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}$.

Related: [4.4 Two Brownian Motions](#)

Show the code (Python + NumPy)

```
import numpy as np
rng, rho, t, n = np.random.default_rng(120), 0.6, 2.0, 2_000_000
Z1, Z2 = rng.normal(size=n), rng.normal(size=n)
W1, W2 = np.sqrt(t) * Z1, np.sqrt(t) * (rho * Z1 + np.sqrt(1 - rho ** 2) * Z2)
print(f"Var(W1 - W2) simulated {(W1 - W2).var():.4f}    formula 2(1 - rho)t = {2 * (1 - rho) * t:.4f}")
```

Output when we ran it:

```
Var(W1 - W2) simulated 1.6007    formula 2(1 - rho)t = 1.6000
```

E · Option pricing

E1 The digital call easy

Price a contract paying one dollar if $S_T > K$ (Black–Scholes).

Hint

Its price is a discounted risk-neutral probability.

Solution

Price = $e^{-rT} Q(S_T > K)$. Under Q , $\ln S_T \sim N(\ln S_0 + (r - \sigma^2/2)T, \sigma^2 T)$, so $Q(S_T > K) = N(d_2)$ with $d_2 = \frac{\ln(S_0/K) + (r - \sigma^2/2)T}{\sigma\sqrt{T}}$. Price = $\boxed{e^{-rT} N(d_2)}$.

Follow-up: a digital is the negative strike-derivative of a call: $-\partial_K C$. Its delta spikes near expiry at the strike, which makes digitals hard to hedge.

Related: [9.2 Greeks by Simulation](#)

Show the code (Python + NumPy)

```
import numpy as np
from math import erf, exp, log, sqrt
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
S0, K, r, sigma, T = 100.0, 110.0, 0.03, 0.2, 1.0
d2 = (log(S0 / K) + (r - sigma ** 2 / 2) * T) / (sigma * sqrt(T))
ST = S0 * np.exp((r - sigma ** 2 / 2) * T + sigma * sqrt(T) *
np.random.default_rng(121).normal(size=4_000_000))
print(f"digital call: Monte Carlo {exp(-r * T) * np.mean(ST > K):.4f}    formula e^(-rT)
N(d2) = {exp(-r * T) * N(d2):.4f}")
```

Output when we ran it:

```
digital call: Monte Carlo 0.3248    formula e^(-rT) N(d2) = 0.3250
```

E2 Asset-or-nothing medium

Price a contract paying S_T if $S_T > K$, and derive the Black–Scholes formula from it.

Hint

Change numeraire: use the stock as the unit of account (a Girsanov shift of σ).

Solution

$e^{-rT} \mathbb{E}_Q[S_T \mathbf{1}_{S_T > K}] = S_0 \mathbb{E}_Q\left[\frac{S_T}{S_0 e^{rT}} \mathbf{1}_{S_T > K}\right] = S_0 Q^S(S_T > K)$, where $\frac{dQ^S}{dQ} = e^{\sigma W_T - \sigma^2 T/2}$.

Under Q^S the drift of $\ln S$ rises by σ^2 , so the probability is $N(d_1)$ with $d_1 = d_2 + \sigma\sqrt{T}$. Price = $S_0 N(d_1)$.

A call is "asset-or-nothing minus K digitals": $C = S_0 N(d_1) - K e^{-rT} N(d_2)$.

Related: [6.2 Girsanov](#)

Show the code (Python + NumPy)

```
import numpy as np
from math import erf, exp, log, sqrt
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
S0, K, r, sigma, T = 100.0, 95.0, 0.03, 0.2, 1.0
d1 = (log(S0 / K) + (r + sigma ** 2 / 2) * T) / (sigma * sqrt(T))
ST = S0 * np.exp((r - sigma ** 2 / 2) * T + sigma * sqrt(T) *
np.random.default_rng(122).normal(size=4_000_000))
print(f"asset-or-nothing: Monte Carlo {exp(-r * T) * np.mean(ST * (ST > K)):.4f}    formula
S0 N(d1) = {S0 * N(d1):.4f}")
```

Output when we ran it:

```
asset-or-nothing: Monte Carlo 69.4065    formula S0 N(d1) = 69.3735
```

E3 The at-the-money rule of thumb easy

Estimate an at-the-money call price quickly (with $r = 0$).

Hint

Expand $N(x) \approx \frac{1}{2} + x \varphi(0)$ for small x .

Solution

At the money with $r = 0$, $d_{1,2} = \pm \frac{1}{2} \sigma \sqrt{T}$, so $C = S(N(\frac{1}{2} \sigma \sqrt{T}) - N(-\frac{1}{2} \sigma \sqrt{T})) \approx S \varphi(0) \sigma \sqrt{T} = \frac{S \sigma \sqrt{T}}{\sqrt{2\pi}} \approx \boxed{0.4 S \sigma \sqrt{T}}$.

Use: it inverts instantly: an at-the-money option trading at price C implies $\sigma \approx 2.5 C / (S \sqrt{T})$. It is accurate to within about 1% for $\sigma \sqrt{T}$ up to 0.4.

Related: [6.1 Black–Scholes by Hedging](#).

Show the code (Python + NumPy)

```
from math import erf, sqrt, pi
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
S, sigma = 100.0, 0.2
for T in (1 / 12, 0.25, 1.0, 4.0):
    # at the money, r = 0
    exact = S * (N(sigma * sqrt(T) / 2) - N(-sigma * sqrt(T) / 2))
    print(f"T = {T:.3f}: exact {exact:.4f}    0.4 S sigma sqrt(T) = {0.4 * S * sigma *
sqrt(T):.4f}    S sigma sqrt(T/2pi) = {S * sigma * sqrt(T / (2 * pi)):.4f}")
```

Output when we ran it:

$T = 0.083$: exact 2.3030	$0.4 S \sigma \sqrt{T} = 2.3094$	$S \sigma \sqrt{T/2\pi} = 2.3033$
$T = 0.250$: exact 3.9878	$0.4 S \sigma \sqrt{T} = 4.0000$	$S \sigma \sqrt{T/2\pi} = 3.9894$
$T = 1.000$: exact 7.9656	$0.4 S \sigma \sqrt{T} = 8.0000$	$S \sigma \sqrt{T/2\pi} = 7.9788$
$T = 4.000$: exact 15.8519	$0.4 S \sigma \sqrt{T} = 16.0000$	$S \sigma \sqrt{T/2\pi} = 15.9577$

E4 Pricing S squared medium

Price a contract paying S_T^2 at T (Black–Scholes).

Hint

Write S_T^2 as an exponential of a normal and use C3.

Solution

$S_T^2 = S_0^2 e^{2(r-\sigma^2/2)T+2\sigma W_T}$, and $\mathbb{E}_Q[e^{2\sigma W_T}] = e^{2\sigma^2 T}$. So $\mathbb{E}_Q[S_T^2] = S_0^2 e^{(2r+\sigma^2)T}$, and the price is e^{-rT} times that: $S_0^2 e^{(r+\sigma^2)T}$.

Follow-up: power claims depend on volatility even though they have no strike: the payoff is convex, and convexity is worth money (Jensen). By Itô, S^2 grows at rate $2r + \sigma^2$.

Related: [4.2 Itô's Lemma](#)

Show the code (Python + NumPy)

```
import numpy as np
from math import exp, sqrt
S0, r, sigma, T = 100.0, 0.03, 0.2, 1.0
ST = S0 * np.exp((r - sigma ** 2 / 2) * T + sigma * sqrt(T) *
np.random.default_rng(124).normal(size=4_000_000))
print(f"price of S_T^2: Monte Carlo {exp(-r * T) * np.mean(ST ** 2):.2f}    formula S0^2
e^((r + sigma^2)T) = {S0 ** 2 * exp((r + sigma ** 2) * T):.2f}")
```

Output when we ran it:

```
price of S_T^2: Monte Carlo 10722.13    formula S0^2 e^((r + sigma^2)T) = 10725.08
```

E5 Parity and early exercise medium

State put–call parity. Why is an American call on a non-dividend stock never exercised early?

Hint

Compare C with $S - Ke^{-rT}$ and with $S - K$.

Solution

A long call plus a short put pays $S_T - K$, the same as a forward, so $C - P = S - Ke^{-r(T-t)}$. Since $P \geq 0$, $C \geq S - Ke^{-r(T-t)} > S - K$ whenever $r > 0$. The option is always worth more than exercising it, so early exercise is never optimal.

Follow-up: for puts the inequality goes the other way. Deep in the money, receiving K now and earning interest beats waiting, so American puts *are* exercised early (9.4). Dividends can make early exercise of calls optimal just before an ex-dividend date.

Related: [9.4 American Options](#)

Show the code (Python + NumPy)

```
from math import erf, exp, log, sqrt
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
S, K, r, sigma, T = 100.0, 105.0, 0.04, 0.25, 0.75
d1 = (log(S / K) + (r + sigma ** 2 / 2) * T) / (sigma * sqrt(T)); d2 = d1 - sigma * sqrt(T)
call, put = S * N(d1) - K * exp(-r * T) * N(d2), K * exp(-r * T) * N(-d2) - S * N(-d1)
print(f"call - put = {call - put:.6f}    S - K e^(-rT) = {S - K * exp(-r * T):.6f}")
print(f"call {call:.4f} >= S - K e^(-rT) = {S - K * exp(-r * T):.4f} > S - K = {S - K:.4f}: never exercise early")
```

Output when we ran it:

```
call - put = -1.896781    S - K e^(-rT) = -1.896781
call 7.7865 >= S - K e^(-rT) = -1.8968 > S - K = -5.0000: never exercise early
```

E6 Why delta is $N(d_1)$ medium

Show that $\partial C / \partial S = N(d_1)$, not $N(d_1)$ plus correction terms.

Hint

Differentiate $C = SN(d_1) - Ke^{-rT}N(d_2)$ and use an identity linking $\varphi(d_1)$ and $\varphi(d_2)$.

Solution

$\partial_S C = N(d_1) + S\varphi(d_1)\partial_S d_1 - Ke^{-rT}\varphi(d_2)\partial_S d_2$. Since $d_2 = d_1 - \sigma\sqrt{T}$, $\partial_S d_1 = \partial_S d_2$. Squaring d_2 shows $S\varphi(d_1) = Ke^{-rT}\varphi(d_2)$, so the last two terms cancel: $\Delta = N(d_1)$.

Interpretation: $N(d_2)$ is the risk-neutral probability of exercise. $N(d_1)$ is the exercise probability under the stock measure (E2), and it is larger because the stock is used as the unit.

Related: [6.1 Black–Scholes by Hedging](#).

Show the code (Python + NumPy)

```
from math import erf, exp, log, sqrt
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
def call(S, K=100.0, r=0.03, sigma=0.2, T=1.0):
    d1 = (log(S / K) + (r + sigma ** 2 / 2) * T) / (sigma * sqrt(T))
    return S * N(d1) - K * exp(-r * T) * N(d1 - sigma * sqrt(T)), N(d1), N(d1 - sigma *
sqrt(T))
h = 1e-4
fd = (call(100 + h)[0] - call(100 - h)[0]) / (2 * h)
print(f"finite-difference delta {fd:.6f}    N(d1) = {call(100)[1]:.6f}    N(d2) = {call(100)
[2]:.6f}")
```

Output when we ran it:

```
finite-difference delta 0.598706    N(d1) = 0.598706    N(d2) = 0.519939
```

F · Greeks and volatility

F1 The P&L of a delta hedge hard

You sell an option at implied volatility σ_{imp} and delta-hedge it. The stock actually moves with volatility σ_{real} . What is your profit and loss?

Hint

Use the Black–Scholes PDE $\Theta + \frac{1}{2}\sigma_{\text{imp}}^2 S^2 \Gamma = 0$ (with $r = 0$) and Itô's lemma with the realized volatility.

Solution

The option (long) changes by $dV = \Theta dt + \Delta dS + \frac{1}{2}\Gamma S^2 \sigma_{\text{real}}^2 dt$. The hedge removes ΔdS . Substituting $\Theta = -\frac{1}{2}\sigma_{\text{imp}}^2 S^2 \Gamma$ gives

$$d(\text{long option} - \Delta S) = \frac{1}{2}\Gamma S^2 (\sigma_{\text{real}}^2 - \sigma_{\text{imp}}^2) dt.$$

Being long gamma earns money when realized volatility beats implied. The seller loses the same amount.

Follow-up: the P&L is path-dependent, since it is weighted by ΓS^2 along the path. Variance swaps (10.3) remove this dependence.

Related: [10.3 Variance Swaps](#)

Show the code (Python + NumPy)

```
import numpy as np
from math import erf, exp, log, sqrt, pi
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
S0, K, T, sig_imp, sig_real, steps = 100.0, 100.0, 1.0, 0.2, 0.3, 2000 # r = 0
def bs(S, tau):
    d1 = (np.log(S / K) + sig_imp ** 2 / 2 * tau) / (sig_imp * np.sqrt(tau))
    return S * N_vec(d1) - K * N_vec(d1 - sig_imp * np.sqrt(tau)), N_vec(d1), np.exp(-d1 **
2 / 2) / np.sqrt(2 * pi) / (S * sig_imp * np.sqrt(tau))
N_vec = np.vectorize(N)
rng, dt, S = np.random.default_rng(127), T / steps, S0
V, pnl, predicted = bs(S0, T)[0], 0.0, 0.0
for i in range(steps): # sell the option at implied
    vol, delta-hedge daily
    price, delta, gamma = bs(S, T - i * dt)
    S_new = S * np.exp(-sig_real ** 2 / 2 * dt + sig_real * np.sqrt(dt) * rng.normal())
    pnl += delta * (S_new - S); predicted += 0.5 * gamma * S ** 2 * (sig_real ** 2 -
sig_imp ** 2) * dt
    S = S_new
pnl = V + pnl - max(S - K, 0)
print(f"short option + delta hedge P&L {pnl:.3f}    predicted -1/2 sum Gamma S^2 (sig_real^2
- sig_imp^2) dt = {-predicted:.3f}")
```

Output when we ran it:

```
short option + delta hedge P&L -5.843    predicted -1/2 sum Gamma S^2 (sig_real^2 -
sig_imp^2) dt = -6.414
```

F2 Vega medium

Compute the vega $\partial C / \partial \sigma$ of a Black–Scholes call and approximate it at the money.

Hint

Use the same $\varphi(d_1) - \varphi(d_2)$ identity as in E6.

Solution

$\partial_\sigma C = S\varphi(d_1)\partial_\sigma d_1 - Ke^{-rT}\varphi(d_2)\partial_\sigma d_2 = S\varphi(d_1)(\partial_\sigma d_1 - \partial_\sigma d_2) = \boxed{S\sqrt{T}\varphi(d_1)}$. At the money, $\varphi(d_1) \approx \varphi(0) \approx 0.4$, so vega $\approx 0.4 S\sqrt{T}$ per unit of volatility.

Follow-up: vega is the same for calls and puts (by parity), peaks near the money, and grows like $\sqrt{T}$. Gamma, by contrast, peaks at short maturities.

Related: [9.2 Greeks by Simulation](#)

Show the code (Python + NumPy)

```
from math import erf, exp, log, sqrt, pi
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
def call(S, sigma, K=100.0, r=0.0, T=1.0):
    d1 = (log(S / K) + (r + sigma ** 2 / 2) * T) / (sigma * sqrt(T))
    return S * N(d1) - K * exp(-r * T) * N(d1 - sigma * sqrt(T)), d1
S, sigma, T, h = 100.0, 0.2, 1.0, 1e-5
fd = (call(S, sigma + h)[0] - call(S, sigma - h)[0]) / (2 * h)
d1 = call(S, sigma)[1]
print(f"vega: finite difference {fd:.4f}    S sqrt(T) phi(d1) = {S * sqrt(T) * exp(-d1 ** 2 / 2) / sqrt(2 * pi):.4f}    0.4 S sqrt(T) = {0.4 * S * sqrt(T):.4f}")
```

Output when we ran it:

```
vega: finite difference 39.6953    S sqrt(T) phi(d1) = 39.6953    0.4 S sqrt(T) = 40.0000
```

F3 Why the equity skew slopes down medium

Why is implied volatility higher for low strikes than high strikes on equity indices?

Hint

What kind of move does a low-strike put protect against, and how does volatility behave then?

Solution

Three reinforcing reasons. (1) **Crash risk:** large downward jumps (8.1) fatten the left tail, so low-strike options are worth more than a lognormal allows (10.1). (2) **Leverage effect:** volatility tends to rise when prices fall ($\rho < 0$ in Heston, 10.2), which also fattens the left tail. (3) **Demand:** investors pay up for downside protection.

Follow-up: jump models produce steep short-dated skews that flatten with maturity. Stochastic volatility produces more persistent skews. Real markets need both.

Related: [10.1 Implied Volatility](#)

Show the code (Python + NumPy)

```
import numpy as np
from math import erf, exp, log, sqrt
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
def bs(S, K, T, r, s):
    d1 = (log(S / K) + (r + s * s / 2) * T) / (s * sqrt(T)); return S * N(d1) - K * exp(-r
    * T) * N(d1 - s * sqrt(T))
def iv(p, S, K, T, r, lo=1e-4, hi=3.0):
    for _ in range(80):
        m = (lo + hi) / 2; lo, hi = (m, hi) if bs(S, K, T, r, m) < p else (lo, m)
    return (lo + hi) / 2
rng, n, S0, T, r, s = np.random.default_rng(129), 2_000_000, 100.0, 0.5, 0.0, 0.15
jumps = rng.poisson(1.0 * T, n) # one crash-like jump a year,
-10% on average
logJ = -0.10 * jumps + 0.10 * np.sqrt(jumps) * rng.normal(size=n)
k = exp(-0.10 + 0.005) - 1
ST = S0 * np.exp((r - 1.0 * k - s * s / 2) * T + s * sqrt(T) * rng.normal(size=n) + logJ)
for K in (85, 100, 115):
    print(f"K = {K}: implied vol {iv(np.maximum(ST - K, 0).mean(), S0, K, T, r):.4f}")
```

Output when we ran it:

```
K = 85: implied vol 0.2221
K = 100: implied vol 0.1957
K = 115: implied vol 0.1818
```

F4 The variance swap strike hard

Show that under Black–Scholes the fair variance-swap strike is σ^2 , using a log contract.

Hint

Itô on $\ln S$: $d \ln S = dS/S - \frac{1}{2}\sigma^2 dt$.

Solution

Integrating, $\frac{1}{T} \int_0^T \sigma^2 dt = \frac{2}{T} \left(\int_0^T dS/S - \ln(S_T/S_0) \right)$. With $r = 0$, $\int dS/S$ is a martingale with mean 0, and $\mathbb{E}_Q[\ln(S_T/S_0)] = -\frac{1}{2}\sigma^2 T$. So the fair strike is $\boxed{\sigma^2}$.

Follow-up: with stochastic volatility the same identity holds pathwise. The log contract is replicated by options weighted by $1/K^2$ (10.3), which is the model-free VIX formula.

Related: [10.3 Variance Swaps](#)

Show the code (Python + NumPy)

```
import numpy as np
from math import sqrt
rng, sigma, T, days, n = np.random.default_rng(130), 0.25, 1.0, 252, 200_000
dt = T / days
lnS = np.cumsum(-sigma ** 2 / 2 * dt + sigma * sqrt(dt) * rng.normal(size=(n, days)),
axis=1) # r = 0
log_contract = 2 / T * (np.exp(lnS[:, -1]) - 1 - lnS[:, -1]) # 2/T (S_T/S0 - 1 - ln(S_T/S0))
print(f"fair variance from the log contract {log_contract.mean():.5f}    sigma^2 = {sigma ** 2:.5f}")
```

Output when we ran it:

```
fair variance from the log contract 0.06254    sigma^2 = 0.06250
```